

DeepLOB–DLS Asset Allocation Module

2026

Complete repository documentation, model description, objective function, execution logic, results, and dashboard guide

Notebook: `deeplob-dls-version1.ipynb`

Branch module: `Asset Allocation Module - DeepLOB + DLS`

Team T001

July 2026

Abstract

This document is a complete version of the DeepLOB-DLS asset allocation documentation. It combines the structure and academic style of the earlier Feishu-style report with the practical repository-level information from the latest README and the notebook-specific technical content from the previous documentation report. The report is written to serve both as a research report and as project documentation for the **LOB-Market-Making** repository.

The implemented pipeline uses a pretrained DeepLOB signal model and a learned DLS portfolio allocation model. DeepLOB is responsible for transforming limit order book and OHLCV features into three directional probabilities for each stock: p_{down} , p_{flat} , and p_{up} . DLS does not replace DeepLOB; instead, it consumes DeepLOB's probability panel over time and learns target portfolio weights. The final target weights are then sent to an execution and backtesting engine that applies real trading constraints such as tradability masks, lot-size restrictions, transaction costs, cash buffers, minimum holdings, target gross exposure, and the no-lookahead execution convention.

The latest executed notebook is a Kaggle Edition workflow. It searches required inputs under `/kaggle/input`, writes artifacts under `/kaggle/working/shifu_dls_oos/`, and packages final outputs into `/kaggle/working/shifu_dls_oos_results.zip`. The executed run uses an initial capital of RMB 50,000,000, evaluates 242 out-of-sample days from D485 to D726, and selects DLS seed 33 from ten candidate seeds. The final DeepLOB + DLS portfolio reaches about RMB 73.05M, corresponding to 46.11% total return, 46.11% CAGR, 1.20 annualized Sharpe ratio, and -15.77% maximum drawdown. Compared with the original DeepLOB-only exact baseline, DLS improves total return from 40.88% to 46.11%, improves Sharpe from 0.96 to 1.20, reduces maximum drawdown from -24.91% to -15.77%, and reduces average holdings from 692.7 to 235.3.

A key point of this documentation is that the DLS loss and execution logic are described according to the actual code in the latest notebook rather than only the markdown narrative. In this version, the DLS loss contains Sharpe, Sortino, drawdown, turnover, concentration, inventory, explicit commission, and optional sparsity terms. The sparsity coefficient is set to zero in the final configuration. No quadratic impact penalty is active in this notebook version.

Contents

1	Executive Summary	5
1.1	What this module adds	5
1.2	Why DLS is needed after DeepLOB	5
1.3	Latest result snapshot	6
2	Repository-Level Documentation	7
2.1	Added files and folders	7
2.2	Notebook path and role	7
2.3	Documentation and dashboard paths	7
2.4	Kaggle workflow	8
3	Data and Runtime Setup	8
3.1	Required input files	8
3.2	Data summary	9
4	Feature Engineering	9
4.1	Intraday LOB grid	9
4.2	Order-flow imbalance formula	9
4.3	Causal OFI normalization	10
4.4	Daily OHLCV feature block	10
5	DeepLOB Signal Network	11
5.1	Role of DeepLOB	11
5.2	Input shape	11
5.3	Architecture	12
6	DLS Portfolio Allocation Network	13
6.1	Feature construction from DeepLOB outputs	13
6.2	Temporal DLS input	13
6.3	Flattening and LSTM	13
7	No-Lookahead Timing Convention	15
7.1	Signal, trade, and label days	15
7.2	Execution timing fix	15
8	DLS Objective Function Verified from Code	16
8.1	Why the objective is portfolio-level	16
8.2	Raw weights, masks, and volatility-scaled weights	16
8.3	Gross return, turnover, and net return	17

8.4	Sharpe reward	17
8.5	Sortino reward	18
8.6	Drawdown penalty	18
8.7	Turnover penalty	19
8.8	Concentration penalty	19
8.9	Inventory / scaled gross exposure penalty	19
8.10	Explicit commission penalty	20
8.11	Optional sparsity penalty	20
8.12	Complete loss in the latest notebook	21
8.13	Important clarification: no quadratic impact in this notebook	21
8.14	Source mapping for loss terms	22
9	Training Procedure and Seed Search	23
9.1	Training setup	23
9.2	Seed search	23
10	Target Weight Post-Processing	24
10.1	Signal-quality filter	25
11	Execution Engine	25
11.1	Purpose	25
11.2	Portfolio value and current weights	26
11.3	Transaction costs	27
11.4	Sell-before-buy sequencing	27
11.5	Lot-size and partial-fill logic	27
11.6	Audit outputs	28
12	Evaluation Metrics	28
13	Out-of-Sample Results	28
13.1	Final DLS results	28
13.2	Baseline comparison	29
13.3	Static backtest diagnostics	30
13.4	Weight distribution diagnostics	31
14	Generated Outputs	32
15	Dashboard Documentation	33
15.1	Purpose	33
15.2	Displayed information	33
15.3	Dashboard equity normalization	34

15.4 Running the dashboard	34
16 Validation and Audit Logic	34
17 Technical Interpretation	35
17.1 How stocks are selected	35
17.2 Why average holdings decrease	35
17.3 Why turnover and costs increase	35
17.4 Why maximum drawdown improves	36
18 Limitations and Future Work	36
18.1 Current limitations	36
18.2 Potential improvements	36
19 Conclusion	36
A Appendix A: Main CONFIG Values	37
B Appendix B: Quick Usage Checklist	38

1 Executive Summary

1.1 What this module adds

This branch adds an **Asset Allocation** module to the LOB-Market-Making repository. The module converts DeepLOB predictions into a complete portfolio strategy. The final workflow is:

```
LOB + OHLCV Features
  -> DeepLOB Signal Model
  -> Down / Flat / Up Probabilities
  -> DLS Feature Panel
  -> DLS Portfolio Allocation Model
  -> Target Portfolio Weights
  -> Execution / Backtest Engine
  -> Portfolio Metrics + Dashboard Replay
```

DeepLOB is used as a signal generator. DLS is used as the portfolio allocation model. The execution engine turns theoretical target weights into realistic trades. The dashboard reconstructs the trading process and makes the output auditable.

1.2 Why DLS is needed after DeepLOB

DeepLOB produces class probabilities, but class probabilities are not the same as portfolio weights. A probability signal still needs to answer several economic questions:

1. Which stocks should be held today?
2. How much capital should be assigned to each stock?
3. How much should the portfolio rebalance relative to yesterday?
4. How much transaction cost is acceptable?
5. How should the strategy control drawdown, turnover, and concentration?

The DLS model addresses these questions by learning an allocation rule from the history of DeepLOB probability signals. Rather than manually applying a fixed top-N or threshold rule, DLS learns a softmax portfolio weight vector and is trained with a differentiable portfolio-level objective.

1.3 Latest result snapshot

Table 1: Latest executed notebook result snapshot.

Item	Value
Notebook file	deeplob-dls-version1.ipynb
Mode	Kaggle Edition
Initial capital	RMB 50,000,000
Out-of-sample period	D485 to D726
Out-of-sample metric days	242
Selected DLS seed	33
Final DLS value	RMB 73.05M
Total return	46.11%
Annualized Sharpe ratio	1.20
Maximum drawdown	-15.77%
Score proxy	20.41
Average holdings per day	235.3

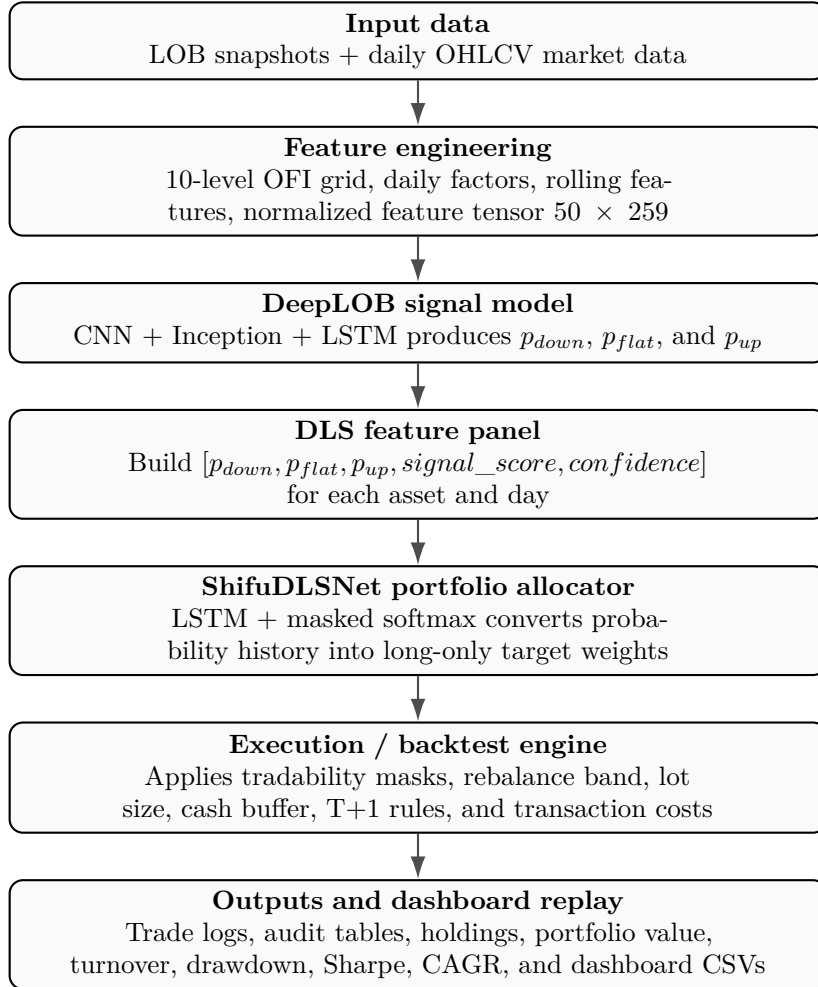


Figure 1: End-to-end DeepLOB + DLS portfolio pipeline.

2 Repository-Level Documentation

2.1 Added files and folders

The README documents the following repository structure for the asset allocation branch:

```
LOB-Market-Making/
|
|-- analysis/
|   '-- notebooks/
|       '-- asset_allocation/
|           '-- deeplob_dls_asset_allocation.ipynb
|
|-- docs/
|   '-- asset_allocation/
|       |-- DeepLOB_DLS_Report_documentation.pdf
|       '-- README.md
|
'-- dashboard/
    '-- asset_allocation/
        |-- app.py
        |-- requirements.txt
        |-- .streamlit/
        |   '-- config.toml
        '-- data/
            '-- exported CSV files
```

2.2 Notebook path and role

The repository path is:

```
analysis/notebooks/asset_allocation/deeplob_dls_asset_allocation.ipynb
```

The executed notebook filename used for the latest documentation update is:

```
deeplob-dls-version1.ipynb
```

This notebook contains the complete DeepLOB + DLS workflow: data loading, LOB feature processing, DeepLOB inference, DLS feature construction, DLS training, seed search, execution, metrics, static charts, audit CSV exports, dashboard preparation, and Kaggle packaging.

2.3 Documentation and dashboard paths

The technical documentation belongs under:

```
docs/asset_allocation/DeepLOB_DLS_Report_documentation.pdf
```

The dashboard application belongs under:

dashboard/asset_allocation/app.py

The dashboard expects exported CSV outputs under:

dashboard/asset_allocation/data/

2.4 Kaggle workflow

The latest notebook is designed as a Kaggle-first workflow. It recursively searches `/kaggle/input` for the required parquet files and model checkpoint. It writes all outputs to:

`/kaggle/working/shifu_dls_oos/`

and creates the downloadable archive:

`/kaggle/working/shifu_dls_oos_results.zip`

The archive size in the executed run is approximately 174.29 MB.

3 Data and Runtime Setup

3.1 Required input files

The notebook requires four parquet market-data files and one pretrained DeepLOB checkpoint.

Table 2: Required inputs.

File	Role
<code>daily_data_in_sample.parquet</code>	Daily OHLCV data used as warmup and training history.
<code>lob_data_in_sample.parquet</code>	In-sample LOB data used to build historical DeepLOB features.
<code>daily_data_release_stage_out_of_sample.parquet</code>	Out-of-sample daily OHLCV data used for final backtesting.
<code>lob_data_release_stage_out_of_sample.parquet</code>	Out-of-sample LOB data used for out-of-sample DeepLOB signals.
<code>best_model_alpha_0015.pt</code>	Pretrained DeepLOB checkpoint.

The executed notebook resolved the paths under:

```
/kaggle/input/datasets/arshiaabolghasemi/feishu-dataset/  
/kaggle/input/models/.../best_model_alpha_0015.pt
```

3.2 Data summary

The executed notebook reports the following data dimensions.

Table 3: Executed notebook data summary.

Item	Value
Number of assets	2,306
Total trading days	726
Warmup / in-sample period	D001 to D484
Out-of-sample period	D485 to D726
Out-of-sample days	242
Combined OHLCV rows	1,606,720
Warmup LOB rows	24,810,697
Out-of-sample LOB rows	12,761,532
Total processed LOB rows	37,572,229
DeepLOB signal days	675
First observation day	D050
First trade day	D051
Assets scored on first observation day	1,876

4 Feature Engineering

4.1 Intraday LOB grid

The LOB data use ten price/volume levels on both the ask and bid sides. The intraday grid contains 24 time slots:

$$09:40, 09:50, \dots, 11:20, 13:00, \dots, 15:00$$

For each asset and each trading day, order-flow imbalance values are placed into a fixed grid of size:

$$N_{slots} \times N_{levels} = 24 \times 10. \quad (1)$$

After flattening, the notebook obtains:

$$N_{OFI} = 240. \quad (2)$$

4.2 Order-flow imbalance formula

For two consecutive snapshots at level ℓ , the bid-side order flow is:

$$bOF_{\tau}^{(\ell)} = \begin{cases} V_{bid,\tau}^{(\ell)}, & P_{bid,\tau}^{(\ell)} > P_{bid,\tau-1}^{(\ell)}, \\ V_{bid,\tau}^{(\ell)} - V_{bid,\tau-1}^{(\ell)}, & P_{bid,\tau}^{(\ell)} = P_{bid,\tau-1}^{(\ell)}, \\ -V_{bid,\tau-1}^{(\ell)}, & P_{bid,\tau}^{(\ell)} < P_{bid,\tau-1}^{(\ell)}. \end{cases} \quad (3)$$

The ask-side order flow is:

$$aOF_{\tau}^{(\ell)} = \begin{cases} -V_{ask,\tau-1}^{(\ell)}, & P_{ask,\tau}^{(\ell)} > P_{ask,\tau-1}^{(\ell)}, \\ V_{ask,\tau}^{(\ell)} - V_{ask,\tau-1}^{(\ell)}, & P_{ask,\tau}^{(\ell)} = P_{ask,\tau-1}^{(\ell)}, \\ V_{ask,\tau}^{(\ell)}, & P_{ask,\tau}^{(\ell)} < P_{ask,\tau-1}^{(\ell)}. \end{cases} \quad (4)$$

The order-flow imbalance is:

$$OFI_{\tau}^{(\ell)} = bOF_{\tau}^{(\ell)} - aOF_{\tau}^{(\ell)}. \quad (5)$$

A positive OFI indicates net buying pressure. A negative OFI indicates net selling pressure.

4.3 Causal OFI normalization

The notebook normalizes each OFI feature with a rolling window of previous days only. With $N_{norm} = 5$:

$$\widehat{OFI}_{f,t} = \frac{OFI_{f,t} - \hat{\mu}_{f,t}}{\hat{\sigma}_{f,t} + \varepsilon}, \quad \hat{\mu}_{f,t}, \hat{\sigma}_{f,t} \text{ computed from } \{t-5, \dots, t-1\}. \quad (6)$$

The current day is excluded from the normalization statistics. This is important because it keeps the OFI feature block causal.

4.4 Daily OHLCV feature block

The OFI block is combined with a daily OHLCV feature block. The daily block has 19 features: 14 engineered features and 5 raw price/volume features. The engineered features are listed in Table 4.

Table 4: Engineered daily OHLCV features.

Feature	Definition	Purpose
ret_1d	$c_t/c_{t-1} - 1$	One-day momentum/reversal
ret_5d	$c_t/c_{t-5} - 1$	One-week momentum
ret_10d	$c_t/c_{t-10} - 1$	Two-week momentum
ret_20d	$c_t/c_{t-20} - 1$	One-month momentum
vol_5d	5-day std of log returns	Short-run realized volatility
vol_20d	20-day std of log returns	Medium-run realized volatility
amihud	$ c_t/c_{t-1} - 1 /(v_t c_t)$	Illiquidity proxy
volume_zscore	20-day volume z-score	Abnormal volume
rsi_14	14-day RSI	Overbought/oversold condition
ma_dist_5	$c_t/SMA_5(c) - 1$	Short moving-average distance
ma_dist_20	$c_t/SMA_{20}(c) - 1$	Medium moving-average distance

Feature	Definition	Purpose
open_close_ret	$c_t/o_t - 1$	Intraday return
high_low_range	$(h_t - l_t)/c_t$	Range / risk proxy
close_vwap_dist	$c_t/vwap_{0930-0935,t} - 1$	Close relative to morning VWAP

The raw daily factors are open, close, volume, low, and high. The daily block is cross-sectionally standardized within each trading day:

$$\tilde{g}_{a,t} = \frac{g_{a,t} - \mu_{g,t}}{\sigma_{g,t} + \varepsilon}, \quad \mu_{g,t}, \sigma_{g,t} \text{ computed across assets on day } t. \quad (7)$$

The final DeepLOB feature width is:

$$N_F = N_{OFI} + N_{OHLCV} = 240 + 19 = 259. \quad (8)$$

5 DeepLOB Signal Network

5.1 Role of DeepLOB

DeepLOB is the signal generation model. It does not directly decide how much capital to allocate. For each asset i and signal day t , DeepLOB outputs:

$$\hat{\mathbf{p}}(t, i) = \begin{bmatrix} p_{down}(t, i) \\ p_{flat}(t, i) \\ p_{up}(t, i) \end{bmatrix} \in \mathbb{R}^3, \quad (9)$$

where:

$$p_{down}(t, i) + p_{flat}(t, i) + p_{up}(t, i) = 1. \quad (10)$$

The interpretation is:

Table 5: DeepLOB probability outputs.

Probability	Meaning
p_{down}	Probability of a price decline.
p_{flat}	Probability of a neutral price movement.
p_{up}	Probability of a price increase.

5.2 Input shape

DeepLOB receives a 50-day window of 259 features:

$$X_{LOB} \in \mathbb{R}^{B \times 1 \times T \times N_F} = \mathbb{R}^{B \times 1 \times 50 \times 259}, \quad (11)$$

where B is the batch size, $T = 50$ is the temporal window length, and $N_F = 259$ is the feature width.

5.3 Architecture

The pretrained DeepLOB architecture contains:

1. convolutional feature extraction blocks;
2. an Inception-style parallel convolution block;
3. a temporal LSTM layer;
4. a fully connected classification head;
5. a softmax output layer.

The loaded checkpoint contains approximately 199,203 trainable parameters. The dummy forward pass gives output shape $(2, 3)$, confirming that the model outputs three class probabilities.

The final softmax can be written as:

$$\hat{p}(t, i) = \text{softmax} \left(W_{DL} h_T^{DL}(t, i) + b_{DL} \right). \quad (12)$$

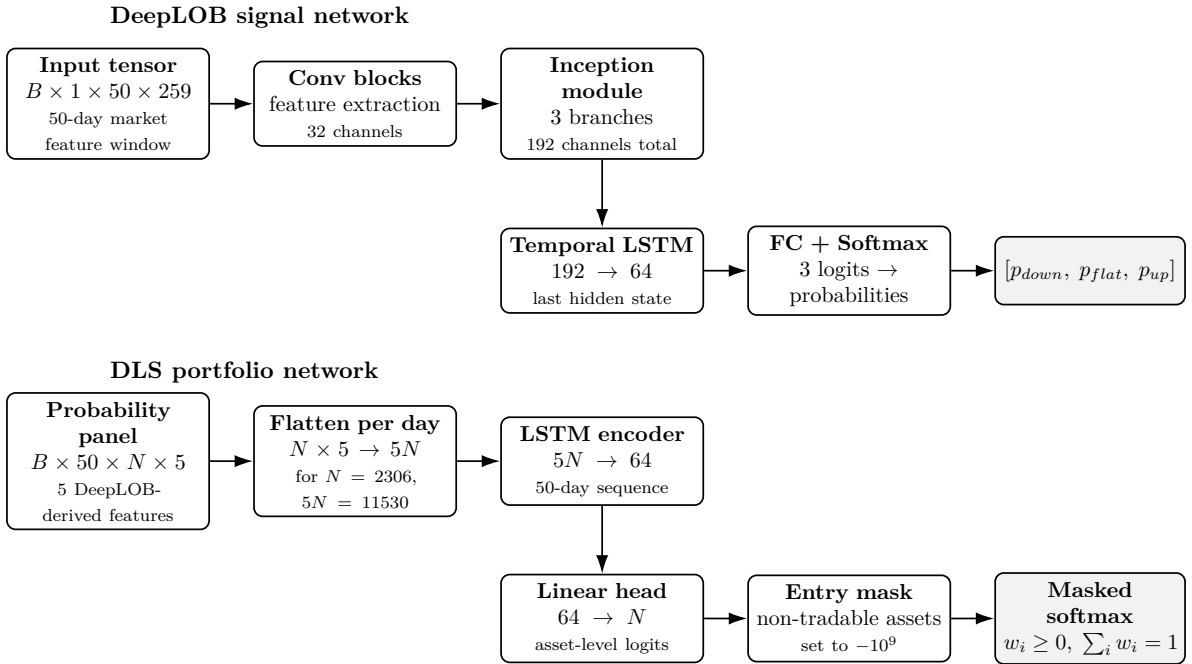


Figure 2: Redrawn neural-network diagrams for the DeepLOB signal model and the DLS portfolio-allocation model.

6 DLS Portfolio Allocation Network

6.1 Feature construction from DeepLOB outputs

DLS receives DeepLOB probability outputs rather than raw LOB tensors. For each asset, the notebook constructs two additional features.

The first one is the directional signal score:

$$\text{signal_score}(t, i) = p_{up}(t, i) - p_{down}(t, i). \quad (13)$$

The second one is the confidence score:

$$\text{confidence}(t, i) = |\text{signal_score}(t, i)| (1 - p_{flat}(t, i)). \quad (14)$$

Therefore, each asset has a five-dimensional DLS feature vector:

$$\mathbf{z}(t, i) = \begin{bmatrix} p_{down}(t, i) \\ p_{flat}(t, i) \\ p_{up}(t, i) \\ \text{signal_score}(t, i) \\ \text{confidence}(t, i) \end{bmatrix} \in \mathbb{R}^5. \quad (15)$$

For the full market:

$$X_{prob} \in \mathbb{R}^{D \times N \times 5}, \quad (16)$$

where D is the number of signal days and N is the number of assets.

6.2 Temporal DLS input

The DLS lookback window is 50 days:

$$L = 50. \quad (17)$$

The DLS input for a signal day t is:

$$X_{DLS}(t) = [X_{prob}(t - L + 1), X_{prob}(t - L + 2), \dots, X_{prob}(t)], \quad (18)$$

with shape:

$$X_{DLS}(t) \in \mathbb{R}^{L \times N \times 5}. \quad (19)$$

6.3 Flattening and LSTM

At each time step, the asset-feature panel is flattened:

$$X_{prob}(t) \in \mathbb{R}^{N \times 5} \longrightarrow \text{vec}(X_{prob}(t)) \in \mathbb{R}^{5N}. \quad (20)$$

In the executed notebook:

$$N = 2306, \quad 5N = 11530. \quad (21)$$

The flattened 50-day sequence is passed into a one-layer LSTM with hidden size 64. The last hidden state is mapped to asset-level logits:

$$\mathbf{a}(t) = W_{DLS} h_L^{DLS}(t) + b_{DLS}, \quad \mathbf{a}(t) \in \mathbb{R}^N. \quad (22)$$

For non-tradable assets, the logit is set to a very negative number before softmax:

$$a_i(t) = -10^9 \quad \text{if asset } i \text{ is not tradable.} \quad (23)$$

Finally, DLS uses masked softmax:

$$w_i(t) = \frac{\exp(a_i(t))}{\sum_{j=1}^N \exp(a_j(t))}. \quad (24)$$

The raw DLS output is long-only:

$$w_i(t) \geq 0, \quad \sum_{i=1}^N w_i(t) = 1. \quad (25)$$

The implemented **ShifuDLSNet** contains approximately 3,118,466 trainable parameters.

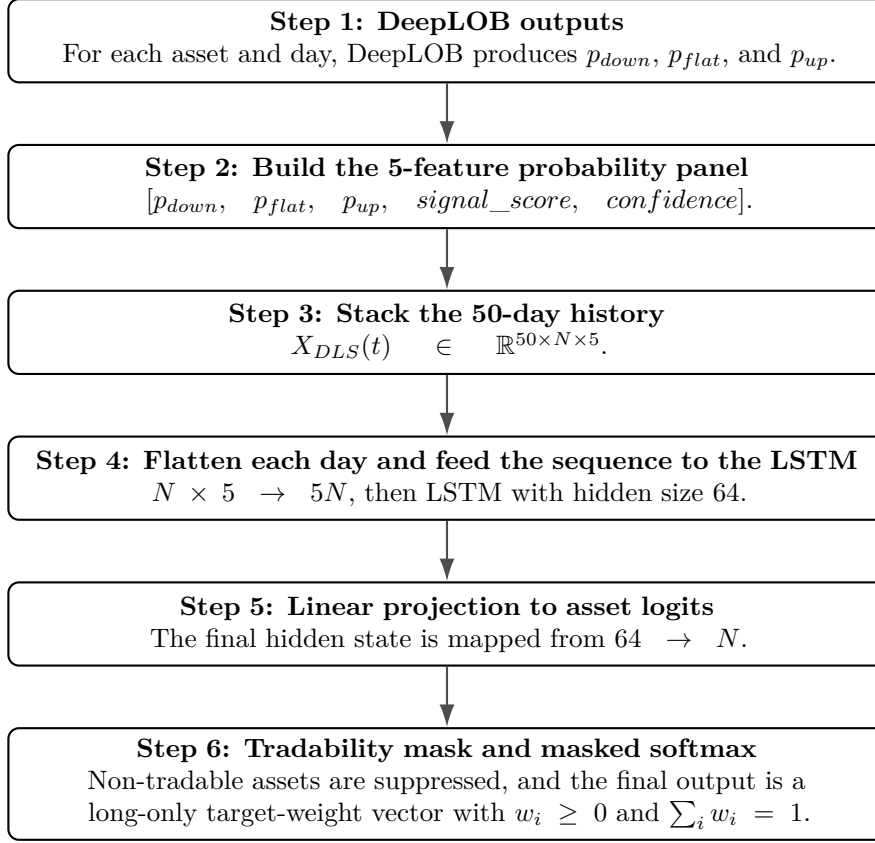


Figure 3: Redrawn DLS architecture and data flow: DeepLOB probabilities are transformed into target portfolio weights.

7 No-Lookahead Timing Convention

7.1 Signal, trade, and label days

The latest notebook uses a shifted no-lookahead convention:

Day	Role
Day t	DeepLOB observes information through the end of the signal day.
Day $t + 1$	Portfolio is traded.
Day $t + 2$	Trade outcome is evaluated using the future close.

The DLS future return label is:

$$R_{future}(t, i) = \frac{Close(t + 2, i)}{VWAP_{entry}(t + 1, i)} - 1. \quad (26)$$

This alignment prevents the allocation decision from using information that would not have been available at decision time.

7.2 Execution timing fix

The final DLS backtest uses a safer execution convention:

- `sell_mode = "open";`
- `decision_price_mode = "open";`
- same-day close is used for end-of-day valuation and metric logging;
- same-day close is not used for same-day execution sizing;
- the shifted label remains $close(t+2)/vwap_{entry}(t+1) - 1$.

This is an important difference between the final DLS version and earlier simple backtest descriptions. It reduces execution-timing bias because trading decisions are not sized from same-day close information.

8 DLS Objective Function Verified from Code

8.1 Why the objective is portfolio-level

DeepLOB is trained as a classifier, but DLS is not trained with a classification loss. DLS is trained with a differentiable portfolio-level objective. This objective asks whether the target weights produced by the network would have generated good net portfolio returns after applying masks, volatility scaling, transaction-cost terms, and risk penalties.

The actual code path is centered on the following functions:

Table 6: Loss-related functions in the notebook code.

Function	Role
<code>paper_net_portfolio_returns</code>	Computes masked, volatility-scaled gross return, turnover, net return, and scaled weights.
<code>sharpe_ratio_torch</code>	Computes the batch Sharpe reward.
<code>sortino_ratio_torch</code>	Computes the downside-risk Sortino reward.
<code>drawdown_penalty_torch</code>	Computes the maximum drawdown penalty from the batch wealth path.
<code>concentration_penalty_torch</code>	Computes average squared raw softmax weights.
<code>inventory_penalty_torch</code>	Penalizes scaled gross exposure away from the target gross.
<code>commission_penalty_torch</code>	Adds an explicit turnover-based commission penalty.
<code>sparsity_penalty_torch</code>	Computes optional sqrt-L1 or entropy sparsity; coefficient is zero in final config.
<code>combined_dls_loss</code>	Combines all terms into the scalar loss minimized by DLS.

8.2 Raw weights, masks, and volatility-scaled weights

For each batch day t , the model first outputs raw long-only softmax weights $w_i(t)$. Before the return is computed, the weights are multiplied by the entry mask and renormalized:

$$\tilde{w}_i(t) = \frac{w_i(t)m_i^{entry}(t)}{\sum_{j=1}^N w_j(t)m_j^{entry}(t) + \varepsilon}. \quad (27)$$

The notebook then applies volatility scaling:

$$scale_i(t) = \frac{\sigma_{target}}{\sigma_i(t) + \varepsilon}, \quad scale_i(t) \leq scale_{max}. \quad (28)$$

The configuration values are:

$$\sigma_{target} = 0.10, \quad scale_{max} = 2.0. \quad (29)$$

The effective scaled weight is:

$$w'_i(t) = \tilde{w}_i(t) scale_i(t). \quad (30)$$

The entry mask is a decision-time mask. The return mask is only used in the gross return calculation to exclude invalid future labels. This separation is important because $close(t+2)$ must not affect whether an asset is considered tradable at decision time.

8.3 Gross return, turnover, and net return

The gross return used in DLS training is:

$$R_{gross}(t) = \sum_{i=1}^N w'_i(t) R_{future}(t, i) m_i^{return}(t). \quad (31)$$

Turnover is computed from the change in scaled weights:

$$TO(t) = \sum_{i=1}^N |w'_i(t) - w'_i(t-1)|. \quad (32)$$

The first previous scaled-weight vector in a chronological batch is either the carried vector from the previous batch or a zero vector. This detail matters because turnover must be consistent across batch boundaries.

The net return is:

$$R_{net}(t) = R_{gross}(t) - c_{train} TO(t), \quad (33)$$

where:

$$c_{train} = \text{dls_cost_rate} = 10^{-4}. \quad (34)$$

Future returns are clipped before training:

$$R_{future}(t, i) \in [-0.70, 0.70]. \quad (35)$$

8.4 Sharpe reward

The code computes:

$$\mu = \mathbb{E}[R_{net}], \quad \sigma^2 = \mathbb{E}[R_{net}^2] - \mu^2. \quad (36)$$

The Sharpe reward is:

$$SR(R_{net}) = \frac{\mu}{\sqrt{\sigma^2 + \varepsilon}}. \quad (37)$$

In the loss, Sharpe enters with a negative sign because the optimizer minimizes loss:

$$\mathcal{L} \supset -SR(R_{net}). \quad (38)$$

8.5 Sortino reward

The code computes downside returns as:

$$R_{down}(t) = \max(-R_{net}(t), 0). \quad (39)$$

The Sortino reward is:

$$Sortino(R_{net}) = \frac{\mathbb{E}[R_{net}]}{\sqrt{\mathbb{E}[R_{down}^2] + \varepsilon}}. \quad (40)$$

Its coefficient is:

$$\lambda_s = 0.25. \quad (41)$$

The loss contribution is:

$$\mathcal{L} \supset -\lambda_s Sortino(R_{net}). \quad (42)$$

8.6 Drawdown penalty

The code first clamps catastrophic returns from below at -0.95 inside the wealth calculation:

$$\overline{R}_{net}(t) = \max(R_{net}(t), -0.95). \quad (43)$$

The wealth path is:

$$Wealth(t) = \prod_{k=1}^t \left(1 + \overline{R}_{net}(k)\right). \quad (44)$$

The running peak is:

$$Peak(t) = \max_{1 \leq k \leq t} Wealth(k). \quad (45)$$

Drawdown is:

$$DD(t) = \frac{Peak(t) - Wealth(t)}{Peak(t) + \varepsilon}. \quad (46)$$

The penalty is the maximum drawdown over the batch:

$$DD = \max_t DD(t). \quad (47)$$

Its coefficient is:

$$\lambda_{dd} = 0.10. \quad (48)$$

8.7 Turnover penalty

The turnover penalty uses the same scaled-weight turnover as the net return cost:

$$\overline{TO} = \mathbb{E}_t[TO(t)]. \quad (49)$$

Its coefficient is:

$$\lambda_{to} = 2.00. \quad (50)$$

This means turnover is penalized twice in the training objective: once directly inside R_{net} through $c_{train}TO(t)$, and once as an explicit penalty through $\lambda_{to}\overline{TO}$. This is intentional in the code and makes DLS more conservative about unnecessary rebalancing.

8.8 Concentration penalty

The concentration penalty is computed on raw softmax weights, not on volatility-scaled weights:

$$C_{conc} = \mathbb{E}_t \left[\sum_{i=1}^N w_i(t)^2 \right]. \quad (51)$$

A larger value indicates that the portfolio is more concentrated in fewer assets. Its coefficient is:

$$\lambda_{conc} = 0.01. \quad (52)$$

8.9 Inventory / scaled gross exposure penalty

The inventory penalty is computed on the volatility-scaled weights:

$$I = \mathbb{E}_t \left[\left(\sum_{i=1}^N |w'_i(t)| - G_{target} \right)^2 \right]. \quad (53)$$

The target gross is:

$$G_{target} = 0.95. \quad (54)$$

Its coefficient is:

$$\lambda_{inv} = 0.02. \quad (55)$$

This term exists because the softmax weights sum to one before volatility scaling, but the scaled effective exposure may differ from the intended gross exposure. The inventory penalty keeps the scaled gross exposure closer to the desired level.

8.10 Explicit commission penalty

The code includes an explicit commission penalty:

$$K = c_{comm} \mathbb{E}_t[TO(t)], \quad (56)$$

where:

$$c_{comm} = 10^{-4}. \quad (57)$$

Its coefficient is:

$$\lambda_{comm} = 2.00. \quad (58)$$

The code comment states that net returns already subtract $dls_cost_rate \times turnover$, but this extra term makes the optimizer more conservative because the realized engine also faces commission, stamp duty, lot-size effects, and cash frictions.

8.11 Optional sparsity penalty

The implementation contains a sparsity term, but the final coefficient is zero:

$$\lambda_{sp} = 0.00. \quad (59)$$

The code supports two alternatives:

$$S_{sqr} = \mathbb{E}_t \left[\sum_{i=1}^N \sqrt{w_i(t) + \varepsilon} \right], \quad (60)$$

or:

$$S_{entropy} = \mathbb{E}_t \left[- \sum_{i=1}^N w_i(t) \log(w_i(t) + \varepsilon) \right]. \quad (61)$$

Because $\lambda_{sp} = 0$, this term is calculated for diagnostics but does not affect the final optimized model. This is consistent with the README note that ordinary L1 regularization is almost constant under long-only softmax weights and earlier sparsity tests reduced holdings too

aggressively.

8.12 Complete loss in the latest notebook

Putting the code terms together, the actual optimized loss is:

$$\mathcal{L}(\theta) = -SR(R_{net}) - \lambda_s \text{Sortino}(R_{net}) + \lambda_{dd} DD(R_{net}) + \lambda_{to} \overline{TO} \quad (62)$$

$$+ \lambda_{conc} C_{conc} + \lambda_{inv} I + \lambda_{comm} K + \lambda_{sp} S. \quad (63)$$

The active coefficient table is:

Table 7: DLS loss coefficients from the latest notebook CONFIG.

Component	Symbol	Value
Sortino reward	λ_s	0.25
Drawdown penalty	λ_{dd}	0.10
Turnover penalty	λ_{to}	2.00
Concentration penalty	λ_{conc}	0.01
Inventory penalty	λ_{inv}	0.02
Commission penalty	λ_{comm}	2.00
Sparsity penalty	λ_{sp}	0.00

8.13 Important clarification: no quadratic impact in this notebook

The latest `deeplob-dls-version1.ipynb` loss does **not** include an active quadratic market-impact term such as:

$$\gamma \mathbb{E}_t [TO(t)^2] \quad \text{or} \quad \gamma \mathbb{E}_t \left[\sum_i (\Delta w_i(t))^2 \right]. \quad (64)$$

Quadratic transaction-cost concepts are relevant background for portfolio optimization, but this specific notebook version uses linear turnover/commission-style terms and not an explicit quadratic impact penalty.

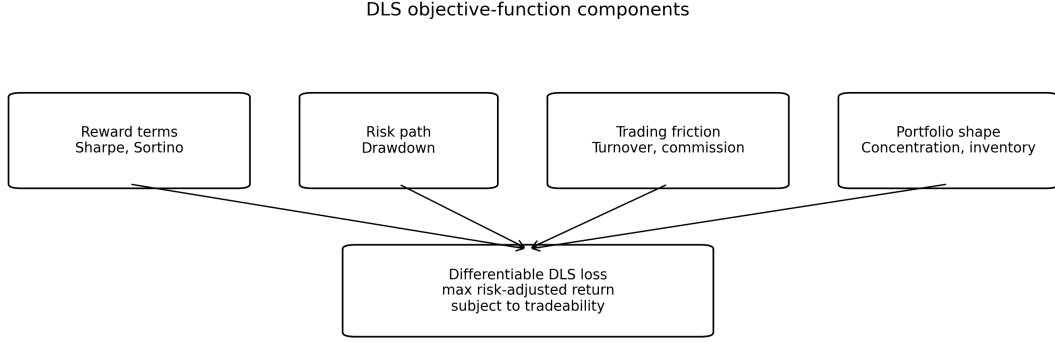


Figure 4: Main DLS loss components. The code uses Sharpe and Sortino as rewards and the other active terms as penalties.

8.14 Source mapping for loss terms

Table 8 maps the loss components to their conceptual source. The formula actually used is the code formula above; the sources explain the financial motivation.

Table 8: Conceptual source mapping for DLS loss terms.

Loss term	Code formula / role	Motivation and source
Sharpe reward	Mean net return divided by net-return volatility.	Standard portfolio risk-adjusted performance measure, consistent with mean-variance portfolio evaluation and the Feishu report’s metric focus.
Sortino reward	Mean net return divided by downside deviation.	Downside-risk extension of Sharpe; rewards return while focusing on negative deviations.
Drawdown penalty	Maximum drawdown from the batch wealth path.	Path-risk control; aligns with the competition’s emphasis on maximum drawdown and practical portfolio risk.
Turnover penalty	Mean $\sum_i w'_i(t) - w'_i(t-1) $.	Dynamic portfolio choice with transaction costs penalizes excessive rebalancing; related to transaction-cost-aware portfolio literature [5, 4].
Concentration penalty	Mean $\sum_i w_i(t)^2$.	Diversification control; discourages allocation collapse into a small number of names.

Loss term	Code formula / role	Motivation and source
Inventory penalty	Mean squared deviation of scaled gross exposure from 0.95.	Gross exposure control and hidden inventory risk management; conceptually connected to cost-aware and dynamic allocation frameworks [4, 7].
Commission penalty	10^{-4} times mean turnover, multiplied by coefficient 2.0.	Explicit linear trading cost; consistent with transaction-cost modeling in portfolio optimization [6, 4].
Sparsity penalty	Sqrt-L1 or entropy option, but inactive because $\lambda_{sp} = 0$.	Internal model-selection regularizer; disabled in final configuration because it reduced holdings too aggressively.

9 Training Procedure and Seed Search

9.1 Training setup

The DLS model is trained chronologically. Temporal order is preserved because turnover depends on consecutive days. The primary training hyperparameters are:

Table 9: DLS training hyperparameters.

Hyperparameter	Value
dls_lookback	50
dls_hidden_size	64
dls_batch_size	64
dls_epochs	100
dls_lr	0.001
dls_early_stopping_enabled	True
dls_early_stopping_patience	15
dls_early_stopping_min_delta	10^{-4}
dls_save_best_val_checkpoint	True

The model is optimized using Adam. The best validation-loss checkpoint is restored before out-of-sample execution.

9.2 Seed search

The notebook trains candidate DLS models with multiple random seeds. The tested seeds are:

7, 11, 22, 33, 42, 55, 77, 88, 101, 123

The selection metric is:

total_return_%

The best seed is:

33

The locked model is saved as:

shifu_dls_model_locked_best_seed.pt

Table 10: Seed-search ranking from the executed notebook.

Seed	Return %	Sharpe	MDD %	Score	Avg holdings
33	46.11	1.202	-15.77	20.41	235.3
7	43.39	1.147	-14.87	19.25	208.6
101	42.83	1.068	-16.55	18.34	231.6
42	39.49	1.024	-15.93	16.86	207.5
77	37.84	0.985	-17.66	15.57	187.2
22	36.42	1.054	-15.31	15.72	254.3
55	31.05	0.870	-19.18	11.79	204.1
11	29.98	0.841	-17.72	11.58	197.8
123	26.97	0.772	-19.78	9.51	219.2
88	25.14	0.736	-21.04	8.26	195.2

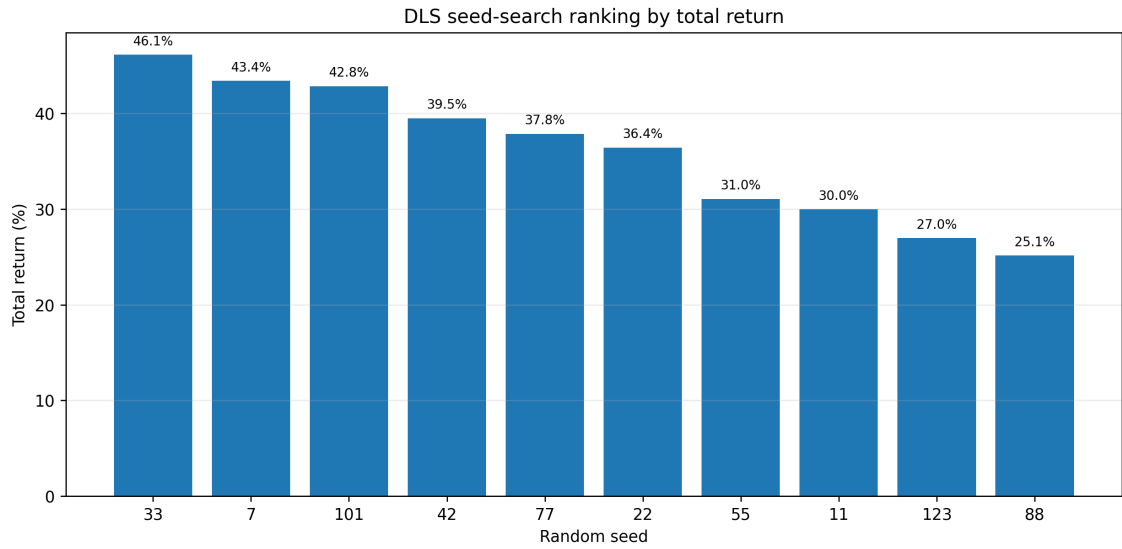


Figure 5: DLS seed-search ranking by total return. Seed 33 is selected.

10 Target Weight Post-Processing

The raw softmax output is not traded directly. It is post-processed before execution:

1. non-tradable stocks are removed;

2. the signal-quality filter is applied;
3. weak weights below the minimum target weight are removed;
4. the maximum number of positions is enforced;
5. the minimum holdings constraint is respected;
6. the final weights are normalized to the target gross exposure.

The final weight normalization is:

$$w_i^{final}(t) = G_{target} \frac{w_i^{filtered}(t)}{\sum_{j=1}^N w_j^{filtered}(t)}, \quad (65)$$

where:

$$G_{target} = 0.95. \quad (66)$$

The key construction parameters are:

Table 11: Post-processing and target-weight parameters.

Parameter	Value
dls_target_gross	0.95
dls_max_positions	500
dls_min_target_weight	0.0005
dls_rebalance_band	0.006
dls_filter_min_candidates	300
dls_filter_fallback	top_confidence

10.1 Signal-quality filter

A stock passes the signal-quality filter only if:

$$\text{confidence}(t, i) \geq 0.02, \quad (67)$$

and:

$$\text{signal_score}(t, i) \geq 0.00. \quad (68)$$

If too few names pass the filter, the notebook falls back to the top-confidence tradable assets. This keeps the strategy selective while still avoiding infeasible portfolios with too few holdings.

11 Execution Engine

11.1 Purpose

The execution engine is the bridge between model output and executable trading. DLS outputs theoretical target weights. The execution engine converts those weights into buy and sell orders

while respecting trading rules, cash limits, lot sizes, costs, and holdings constraints.

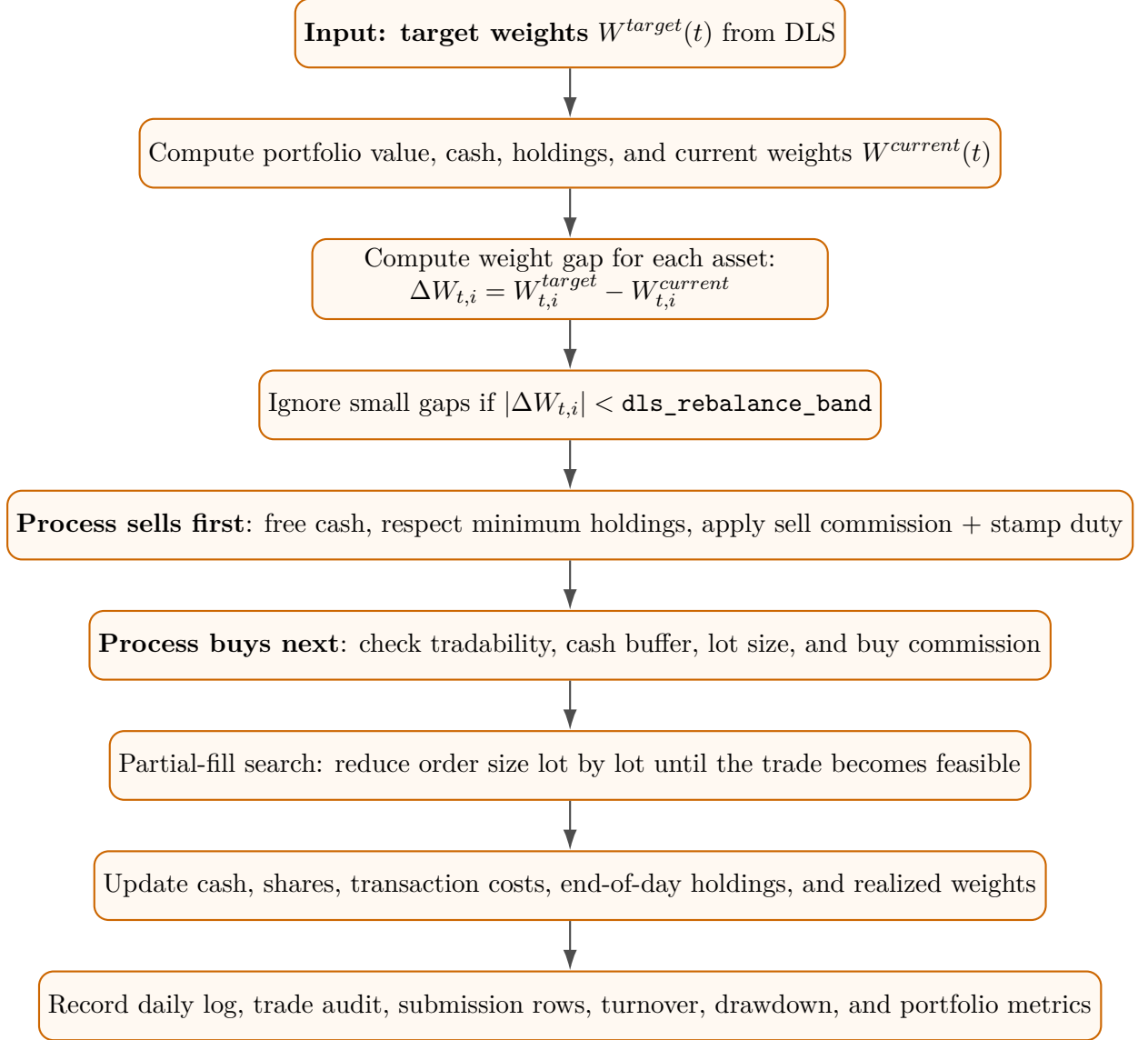


Figure 6: Redrawn execution-engine flowchart. The diagram is drawn directly in $\text{\LaTeX}$ and not inserted as a screenshot.

11.2 Portfolio value and current weights

At each trading day, the engine computes portfolio value:

$$PortfolioValue(t) = Cash(t) + \sum_{i=1}^N shares_i(t) price_i(t). \quad (69)$$

The current weight of asset i is:

$$W_i^{current}(t) = \frac{shares_i(t) price_i(t)}{PortfolioValue(t)}. \quad (70)$$

The weight gap is:

$$\Delta W_{t,i} = W_{t,i}^{target} - W_{t,i}^{current}. \quad (71)$$

If $\Delta W_{t,i} > 0$, the engine tries to buy. If $\Delta W_{t,i} < 0$, the engine tries to sell. If the absolute gap is smaller than the rebalance band, no trade is submitted:

$$|\Delta W_{t,i}| < \text{dls_rebalance_band}. \quad (72)$$

The configured band is 0.006.

11.3 Transaction costs

For a buy trade:

$$BuyCost = \max(BuyValue \times commission_rate, min_commission). \quad (73)$$

For a sell trade:

$$SellCost = \max(SellValue \times commission_rate, min_commission) + SellValue \times stamp_duty_rate. \quad (74)$$

The parameters are:

Table 12: Execution cost and trading-rule parameters.

Parameter	Value
commission_rate	0.0001
stamp_duty_rate	0.0005
min_commission	5
lot_size	100
cash_buffer	0.05
min_holdings	10
sell_mode	open
decision_price_mode	open
use_close_sale_cash_same_day	False

11.4 Sell-before-buy sequencing

The engine processes sells before buys. This is economically useful because selling releases cash that can be used to fund new purchases. However, the latest DLS implementation uses open-price execution conventions and does not rely on same-day close-sale cash for decision-time sizing.

11.5 Lot-size and partial-fill logic

Buy orders must respect the 100-share lot-size rule. If the full target buy is not affordable after commission and available cash constraints, the engine reduces the order size until it finds the largest feasible lot count. If not even one lot is affordable, the buy order is skipped.

11.6 Audit outputs

The execution engine writes multiple audit files, including trade-level, weight-step, and holding-snapshot outputs. These files make it possible to inspect how each signal becomes a target weight and how each target weight becomes an executed order.

12 Evaluation Metrics

The notebook evaluates the out-of-sample portfolio using standard portfolio metrics.

Total return is:

$$TotalReturn = \frac{FinalValue}{InitialCapital} - 1. \quad (75)$$

If the number of metric days is n , and the trading-year convention is 242 days, then:

$$Years = \frac{n}{242}. \quad (76)$$

CAGR is:

$$CAGR = \left(\frac{FinalValue}{InitialCapital} \right)^{1/Years} - 1. \quad (77)$$

Annualized volatility is:

$$AnnualizedVolatility = std(r_{daily})\sqrt{242}. \quad (78)$$

The annualized Sharpe ratio is:

$$Sharpe = \frac{mean(r_{daily} - r_{f,daily})}{std(r_{daily})}\sqrt{242}. \quad (79)$$

Maximum drawdown is:

$$MaxDrawdown = \max_t \left(\frac{Peak(t) - V(t)}{Peak(t)} \right). \quad (80)$$

The unranked score proxy used in the notebook is:

$$ScoreProxy = 0.45 \times CAGR(\%) + 0.30 \times Sharpe \times 10 + 0.25 \times (-MDD(\%)). \quad (81)$$

This is not the official percentile-ranked competition score. It is a raw proxy based on the same three metric types.

13 Out-of-Sample Results

13.1 Final DLS results

The final DeepLOB + DLS model is evaluated over D485 to D726 with 242 metric days.

Table 13: Final DeepLOB + DLS out-of-sample metrics.

Metric	Value
Metric days	242
Dataset share used	33.33%
Initial capital	RMB 50,000,000
Selected DLS seed	33
Final value	RMB 73.05M
Total return	46.11%
CAGR	46.11%
Annualized Sharpe ratio	1.20
Maximum drawdown	-15.77%
Score proxy	20.41
Annualized volatility	35.28%
Calmar ratio	2.92
Total transaction costs	RMB 3,076,075
Average daily turnover	RMB 36,290,607
Win rate	53.72%
Average holdings per day	235.3
Minimum holdings observed	195
Days below minimum holdings	0

The final value calculation is:

$$50,000,000 \times (1 + 0.46108466) \approx 73,054,233. \quad (82)$$

13.2 Baseline comparison

The notebook compares DeepLOB + DLS against the original DeepLOB-only exact baseline.

Table 14: Original DeepLOB-only exact baseline versus DeepLOB + DLS.

Model	Return	CAGR	Sharpe	MDD	Score	Costs	Turnover	Holdings
Original DeepLOB-only exact	40.88%	40.88%	0.96	-24.91%	15.05	RMB 2.10M	RMB 22.94M	692.7
DeepLOB + DLS	46.11%	46.11%	1.20	-15.77%	20.41	RMB 3.08M	RMB 36.29M	235.3

Relative to the original DeepLOB-only exact baseline, DLS achieves:

- +5.23 percentage points higher total return;
- higher Sharpe ratio, from 0.96 to 1.20;
- less severe maximum drawdown, from -24.91% to -15.77%;
- higher score proxy, from 15.05 to 20.41;
- much lower average holdings, from 692.7 to 235.3;
- higher transaction costs and turnover, reflecting a more active and selective reallocation process.

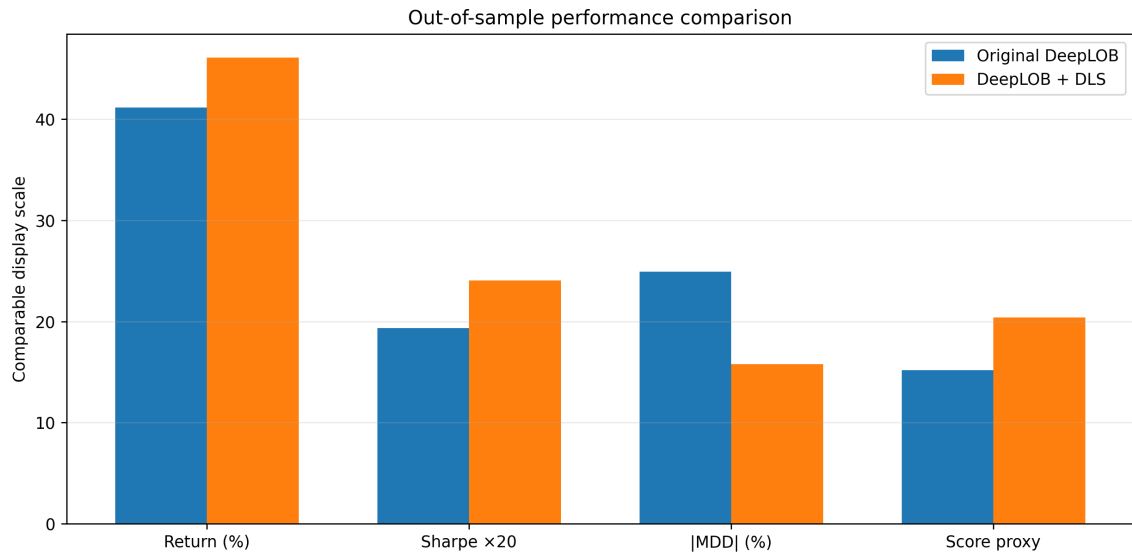


Figure 7: Out-of-sample comparison between the original DeepLOB baseline and DeepLOB + DLS.

13.3 Static backtest diagnostics

The executed notebook saves a static backtest report chart. It includes portfolio value, daily return distribution, drawdown curve, turnover/cost behavior, holdings, and additional diagnostic plots.

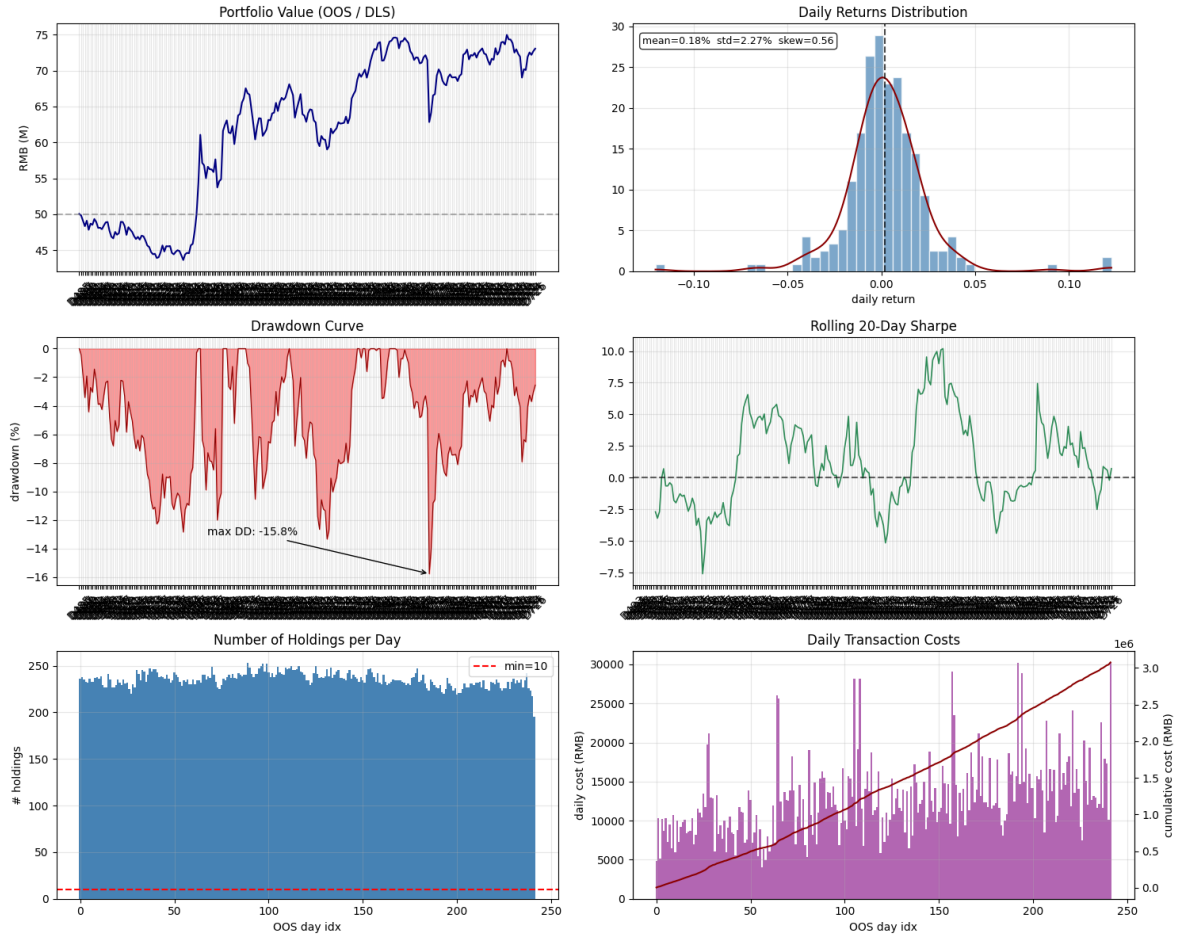


Figure 8: Static DLS backtest report generated by the notebook.

13.4 Weight distribution diagnostics

The executed notebook also saves the distribution of nonzero DLS target weights. The run reports:

Table 15: DLS target-weight distribution summary.

Statistic	Value
Nonzero weights	48,890
Positive weights	48,890
Negative weights	0
Mean	0.004683
Standard deviation	0.010766
Minimum	0.000488
Median	0.001478
95th percentile	0.017994
99th percentile	0.055633
Maximum	0.175141

The absence of negative weights confirms the long-only nature of the current DLS design.

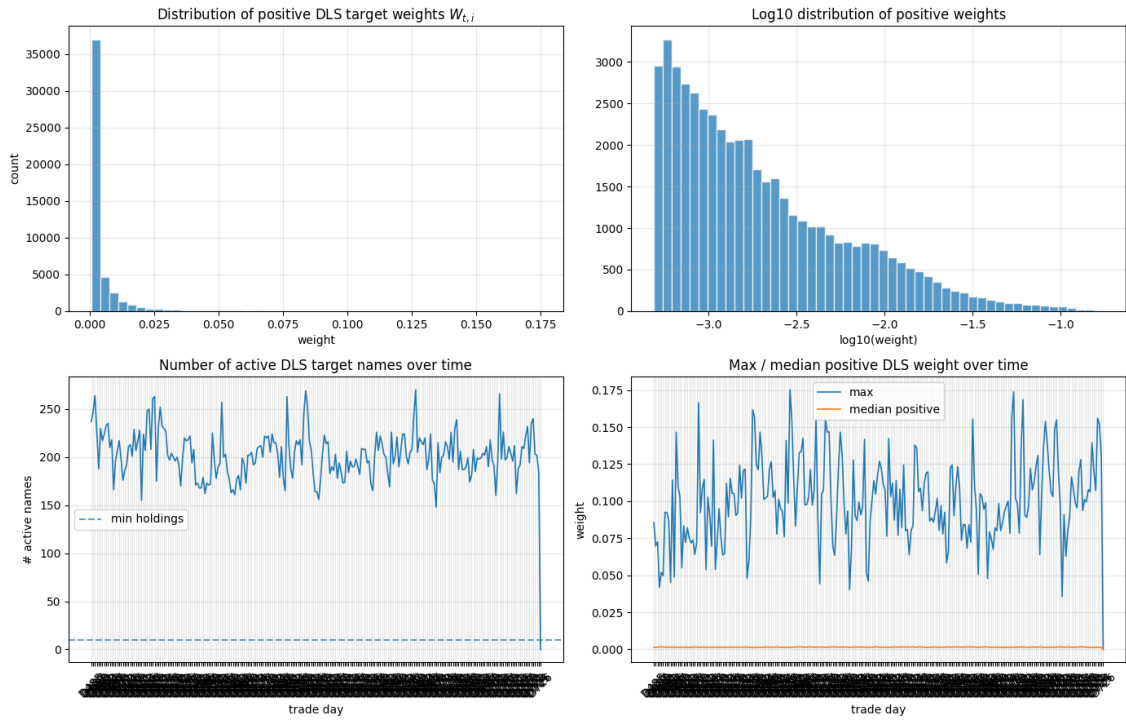


Figure 9: Distribution of DLS target weights and related weight diagnostics.

14 Generated Outputs

The notebook exports model checkpoints, diagnostic CSV files, submissions, comparison tables, and static figures. The main outputs are listed in Table 16.

Table 16: Generated notebook outputs.

Output file	Purpose
shifu_dls_model_locked_best_seed.pt	Final selected DLS checkpoint.
shifu_dls_seed_search_summary.csv	Seed-search summary for all tested seeds.
shifu_dls_training_history_oos.csv	DLS train/validation history.
T001_dls_weight_debug_shifted_tplus2_conf_filter.csv	DLS daily diagnostics.
T001_dls_weights_long_shifted_tplus2.csv	Long-format DLS target weights.
T001_dls_trade_audit.csv	Executed trade audit.
T001_dls_weight_step_audit.csv	Weight-adjustment audit.
T001_dls_holding_snapshot_audit.csv	Holdings snapshot audit.
T001_oos_shifu_dls_kaggle_sell_open.csv	Final Kaggle DLS submission / trade log.
T001_oos_original_deeplob_exact_sell_close.csv	Original DeepLOB-only baseline submission / trade log.
T001_original_deeplob_exact_raw_oos_signals.csv	Baseline raw probabilities and signals.

Output file	Purpose
T001_original_deeplob_exact_daily_log.csv	Baseline daily portfolio log.
T001_comparison_original_deeplob_exact_vs_dls.csv	Baseline comparison table.
backtest_report_shifu_dls_shifted_tplus2_conf_filter.png	Static DLS backtest chart.
dls_weight_distribution_Wt.png	DLS target-weight distribution chart.

The final DLS trade log passes validation and contains:

Table 17: Final trade log summary.

Item	Value
Trade log rows	7,495
Trading days	242
Unique traded assets	459

15 Dashboard Documentation

15.1 Purpose

The Streamlit dashboard provides an interactive replay of the trading engine. It is not just a CSV viewer. It reconstructs the trading process as a replayable state machine:

signals -> filters -> target weights -> orders -> executions ->
 holdings -> portfolio state

15.2 Displayed information

The dashboard visualizes:

- DeepLOB probabilities p_{down} , p_{flat} , and p_{up} ;
- DLS target weights;
- tradability masks;
- signal-score and confidence filters;
- buy and sell orders;
- executed trades;
- holdings;
- transaction costs;

- portfolio value path;
- drawdown behavior;
- turnover behavior;
- comparison between DeepLOB-only and DeepLOB + DLS;
- expanded asset-level audit tables.

15.3 Dashboard equity normalization

The dashboard normalizes the DLS equity replay so that the first valid DLS replay value starts from RMB 50,000,000. This makes visual comparison with the notebook metrics consistent:

$$DLS\ Equity\ Proxy_{raw} = \frac{EOD\ Market\ Value}{Engine\ Target\ Gross}. \quad (83)$$

$$ScaleFactor = \frac{50,000,000}{FirstValid(DLS\ Equity\ Proxy_{raw})}. \quad (84)$$

$$DLS\ Equity\ Proxy = DLS\ Equity\ Proxy_{raw} \times ScaleFactor. \quad (85)$$

15.4 Running the dashboard

From the project root:

```
streamlit run dashboard/asset_allocation/app.py
```

Required Python packages are:

```
streamlit, pandas, numpy, plotly, streamlit-autorefresh
```

16 Validation and Audit Logic

The final submission/trade log is checked before saving. The validation rules include:

1. no duplicated $(trade_day_id, asset_id)$ rows;
2. buy and sell percentages must be between 0 and 1;
3. no zero-action rows are kept;
4. no row can have both buy and sell percentage positive;
5. daily holdings must not fall below `min_holdings`.

The executed notebook prints:

Validation passed Rows: 7495 Days: 242 Assets: 459

The final DLS backtest also reports:

Selected seed : 33 OOS days run : 242 Final value : RMB 73.05M Total return : 46.11% Min holdings : 195 Days < 10 : 0
--

17 Technical Interpretation

17.1 How stocks are selected

Stock selection is not a single hard-coded top-N selection step. It happens through several layers:

1. DeepLOB scores assets using LOB and OHLCV features.
2. DLS receives the probability history and learns allocation weights.
3. Non-tradable assets are masked.
4. The signal-quality filter keeps assets with sufficient confidence and nonnegative signal score.
5. Very small target weights are removed.
6. The number of positions is capped at 500.
7. The minimum-holdings rule is protected.
8. The execution engine only executes feasible trades.

Thus, stock selection is not purely DeepLOB and not purely DLS. It is the result of DeepLOB signals, DLS learned weights, signal/tradability filters, and execution constraints.

17.2 Why average holdings decrease

The original DeepLOB-only baseline holds about 692.7 names on average. The DLS version holds about 235.3. This means DLS is more selective. It concentrates on a smaller set of names while still remaining diversified enough to satisfy the minimum-holdings requirement and the target-gross framework.

17.3 Why turnover and costs increase

DLS increases average daily turnover from RMB 22.94M to RMB 36.29M and total costs from RMB 2.10M to RMB 3.08M. This is the cost of more active reallocation. The important point is that the higher turnover is compensated by better out-of-sample return, Sharpe, drawdown, and score proxy in the executed run.

17.4 Why maximum drawdown improves

The DLS objective directly penalizes drawdown, turnover, concentration, inventory deviation, and commission exposure while rewarding risk-adjusted returns. This makes the portfolio decision more risk-aware than a simple probability-threshold rule. The result is a maximum drawdown improvement from -24.91% to -15.77%.

18 Limitations and Future Work

18.1 Current limitations

The current implementation has several limitations:

1. The DLS output is long-only because it uses softmax weights.
2. The loss uses turnover and commission-style linear cost terms, but no active quadratic impact term in this version.
3. The seed-search selection uses out-of-sample total return as the selection metric; this is useful for development analysis but should be treated carefully in production research.
4. The DLS model flattens the full asset-feature panel into a vector of size $5N$, so the model is tied to the asset universe size used in the run.
5. The strategy is evaluated on one competition OOS period, so robustness across other market regimes remains a further research step.

18.2 Potential improvements

Possible extensions include:

1. Add an explicit quadratic impact penalty for large rebalances.
2. Replace seed selection by a stricter validation-only protocol.
3. Use graph or attention models to capture cross-asset relationships more directly.
4. Add sector/industry neutrality or risk-model constraints if sector data become available.
5. Compare DLS against equal-weight top-confidence, top-N, and mean-variance baselines.
6. Add more robust walk-forward evaluation across multiple OOS windows.
7. Calibrate turnover penalties to realized cost slippage instead of fixed constants.

19 Conclusion

This documentation describes the complete DeepLOB + DLS asset allocation module in a form that is both research-oriented and repository-oriented. The implemented system uses DeepLOB as a signal generator and DLS as a learned portfolio allocator. The DLS model converts a 50-day

history of DeepLOB probability panels into long-only target portfolio weights. Those weights are then post-processed and sent through an execution engine that applies realistic trading rules.

The latest executed notebook shows that the DLS allocation layer improves the economic use of DeepLOB probabilities. Over the 242-day out-of-sample period, DeepLOB + DLS reaches 46.11% total return, 1.20 Sharpe, and -15.77% maximum drawdown. The original DeepLOB-only baseline reaches 40.88% total return, 0.96 Sharpe, and -24.91% maximum drawdown. DLS therefore improves return, risk-adjusted performance, drawdown control, and score proxy, while creating a more selective portfolio with fewer average holdings.

The practical contribution of the branch is not only a model, but a full research-to-execution workflow: data processing, DeepLOB signal generation, DLS allocation, code-verified portfolio loss, transaction-cost-aware execution, output packaging, audit tables, and dashboard replay.

A Appendix A: Main CONFIG Values

Table 18: Main configuration values in the latest notebook.

Parameter	Value
initial_capital	50,000,000
min_holdings	10
sell_mode	open
decision_price_mode	open
use_close_sale_cash_same_day	False
lot_size	100
commission_rate	10^{-4}
stamp_duty_rate	5×10^{-4}
min_commission	5
T	50
NF	259
N_LEVELS	10
N_OFI	240
OFI_NORM_DAYS	5
ALPHA	0.015
min_up_prob	0.75
min_down_prob	0.85
partial_sell_pct	0.50
cash_buffer	0.05
dls_lookback	50
dls_hidden_size	64
dls_epochs	100
dls_lr	0.001
dls_batch_size	64
dls_cost_rate	10^{-4}
dls_return_clip_abs	0.70
dls_sigma_tgt	0.10

Parameter	Value
dls_vol_span	50
dls_max_vol_scale	2.0
dls_target_gross	0.95
dls_max_positions	500
dls_min_target_weight	0.0005
dls_rebalance_band	0.006
dls_min_confidence	0.02
dls_min_signal_score	0.00
dls_filter_min_candidates	300
risk_free_annual	0.0172
trading_days_year	242

B Appendix B: Quick Usage Checklist

1. Attach `arshiaabolghasemi/feishu-dataset` in Kaggle.
2. Attach a model dataset containing `best_model_alpha_0015.pt`.
3. Enable a GPU accelerator if available.
4. Run all notebook cells.
5. Confirm that `Validation passed` is printed.
6. Download `/kaggle/working/shifu_dls_oos_results.zip`.
7. Copy exported CSV files into `dashboard/asset_allocation/data/`.
8. Run `streamlit run dashboard/asset_allocation/app.py` for replay and audit.

References

- [1] Project README, *Asset Allocation Module – DeepLOB + DLS*, latest README markdown supplied with the notebook.
- [2] *Technical Report on Portfolio Management: Notebook Combining DeepLOB and DLS*, documentation report supplied with the project.
- [3] *Research Report for the Feishu Quant Competition 2026: Portfolio Strategy of Team T030*, Feishu reference report.
- [4] Fan, Q., Medeiros, M. C., Yang, H., and Yang, S., *Cost-aware Portfolios in a Large Universe of Assets*, 2025.
- [5] Moallemi, C. C. and Saglam, M., *Dynamic Portfolio Choice with Linear Rebalancing Rules*, 2015.

- [6] Chen, P., Lezmi, E., Roncalli, T., and Xu, J., *A Note on Portfolio Optimization with Quadratic Transaction Costs*, 2019.
- [7] Collin-Dufresne, P., Daniel, K. D., and Saglam, M., *Optimal Dynamic Asset Allocation with Transaction Costs: The Role of Hedging Demands*, 2024.
- [8] Muhle-Karbe, J., Sefton, J., and Shi, X., *Dynamic Portfolio Choice with Intertemporal Hedging and Transaction Costs*, 2025.